

Codex GPT-5.4 Reference vs Claude Code Qwen3.5 9B Evaluation

Generated: 2026-06-19 15:47:50 UTC

Goal

This report evaluates the Claude Code benchmark answer run in ``docs/evaluations/answers/claude_code_qwen35_9b_20260619T102436Z.json`` against the Codex GPT-5.4 reference answers stored in ``docs/evaluations/eval_answers_v2.json``. It explains, question by question, whether the Claude Code answer agrees with the reference answer, partially covers it, or misses the core expected answer.

Test Environment

Codex Reference Side

- Model: ``GPT-5.4 (Codex)``
- Reference file: ``docs/evaluations/eval_answers_v2.json``
- Question source: ``eval_questions_v2.json``
- Evaluation basis: semantic comparison against the Codex reference answer for each question

Claude Code / Ollama Side

- Claude Code model id: ``claude-qwen35-9b``
- Run label: ``qwen35_9b``
- Ollama tag: ``qwen3.5:9b``
- Host: ``merlin-g-100.psi.ch``
- Anthropic-compatible gateway: ``http://127.0.0.1:4000``
- Allowed Claude Code tools: ``Read,Grep,Glob``
- Run started: ``2026-06-19T10:24:36.937906+00:00``
- Run finished: ``2026-06-19T12:00:51.801069+00:00``
- Recorded answer count: ``100``
- Recorded failure count in metadata: ``56``

Important Caveat

The verdicts are a structured comparison against the Codex reference answers, not a fresh manual audit of every IPPL source file. Correct means the Claude Code answer captures the central reference facts; Partial means it contains relevant signal but misses important scope, exact location, or implementation detail; Incorrect means it failed, abstained, pointed to the wrong subsystem, or omitted the core reference answer.

Because Claude Code answers are often much longer than the concise reference answers, the comparison emphasizes central technical nouns, exact identifiers, files, classes, functions, and implementation steps rather than exact wording.

Overall Result

Metric Value
--- ---:
Questions 100
Correct 26
Partial 4
Incorrect 70
Average score (<code>`Correct=1`, <code>`Partial=0.5`, <code>`Incorrect=0`</code></code>) 0.280 </code>
Runtime/API failures counted as incorrect 14
Metadata failure count 56
Answers with leaked self-correction/planning text 8
Mean answer latency 57.75s
Median answer latency 26.19s

Main Findings

- Strongest areas: Testing And Workflow, File Purpose. These categories mostly reward direct repository navigation, API identification, and concise file/class role descriptions.
- Weakest areas: Parallelism And Kokkos, Numerical Meaning. These categories were hurt by failed generations, missing exact implementation locations, shallow source inspection, or cross-file reasoning gaps.
- The report detected ``14`` answer entries with explicit runtime/API failures; the source metadata records ``56`` failures for the full run.
- The Claude Code/Ollama setup had mean latency ``57.75s`` per question, with a median of ``26.19s``.
- ``8`` answers were flagged for leaked planning/self-correction text, such as step-by-step source-inspection narration.

Category Breakdown

Category	Questions	Correct	Partial	Incorrect	Avg score
File Purpose	13	8	0	5	0.615
Definition Location	15	4	3	8	0.367
Class Responsibility	13	4	0	9	0.308
Algorithm	12	2	0	10	0.167
Data Flow	9	1	1	7	0.167
Api Usage	12	2	0	10	0.167
Parallelism And Kokkos	9	0	0	9	0.000
Boundary And Halo	5	1	0	4	0.200
Numerical Meaning	3	0	0	3	0.000
Build And Install	5	0	0	5	0.000
Testing And Workflow	4	4	0	0	1.000

Detailed Per-Question Evaluation

File Purpose

file_001 - What does Ippl.h do?

- Verdict: ****Correct****
- Correct reference answer: Declares the top-level IPPL runtime handles (``Comm``, ``Env``, ``Info``, ``Warn``, ``Error``) and the lifecycle wrappers ``initialize``, ``finalize``, ``fence``, and ``abort``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.64``
- Reference identifier/location hits: ``8/9``
- Latency: ``66.00s``
- Matched reference signals: ``comm, env, info, warn, error, wrappers, initialize, finalize``
- Missing/weak reference signals: ``top-level, runtime, handles, lifecycle, abort``
- Claude Code answer excerpt: ``on my analysis of Ippl.h, here's what it does: ## `Ippl.h` – The Core IPPL Environment Header File: `/psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/src/Ippl.h` (49 lines) Purpose It defines the global parallel communication environment for IPPL, a C++ library for massively parallel co...``

file_002 - What is the purpose of Ippl.cpp?

- Verdict: ****Incorrect****
- Correct reference answer: Implements runtime setup/teardown: command-line option parsing, MPI communicator/environment creation, logger setup, Kokkos init/finalize, and abort/fence helpers.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Latency: ``7.25s``
- Missing/weak reference signals: ``implements, runtime, setup/teardown, command-line, option, parsing, mpi, communicator/environment``
- Claude Code answer excerpt: ``'ll explore the repository to find and understand `Ippl.cpp` – let's start by locating it:``

file_003 - What does FFT.hpp implement?

- Verdict: ****Incorrect****
- Correct reference answer: Implements ``FFTBase`` setup plus the FFT specialization family for complex/real, sine, cosine, and cosine-I transforms on IPPL fields via `heFFTe`.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: ``0.00``
- Latency: ``115.71s``
- Recorded error: ``exit 1``
- Missing/weak reference signals: ``implements, fftbase, setup, fft, specialization, family, complex/real, sine``
- Claude Code answer excerpt: ``API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE_CODE_MAX_OUTPUT_TOKENS environment variable.``

file_004 - What is BareField.h responsible for?

- Verdict: ****Correct****
- Correct reference answer: Defines the core distributed field container: Kokkos-backed storage, owned/allocated domains, field layout access, halo exchange hooks, and expression-style assignment.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.50``
- Latency: ``33.53s``
- Matched reference signals: ``core, distributed, field, layout, access, halo, assignment``
- Missing/weak reference signals: ``container, kokkos-backed, storage, owned/allocated, domains, exchange, hooks,`

expression-style`
- Claude Code answer excerpt: `on examining `BareField.h` and `BareField.hpp`, here's what **``BareField.h``** is responsible for: **## Main Responsibilities of BareField Header Files** **### 1. ``Declaration of the `BareField` class template``** (`BareField.h`) - Declares a templated field representation supporting multiple data types and dimensions (curr...`

file_005 - What does Field.h add on top of BareField?

- Verdict: **``Correct``**
- Correct reference answer: Adds mesh awareness and boundary-condition management on top of `BareField`, plus volume integral/average helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Reference identifier/location hits: `1/1`
- Latency: `19.10s`
- Matched reference signals: `mesh, management, barefield, volume, BareField`
- Missing/weak reference signals: `awareness, boundary-condition, integral/average, helpers`
- Claude Code answer excerpt: `on my analysis of the code in `/psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/src/Field/BareField.hpp` and `/psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/src/Field/Field.h`, here's what `Field.h` adds on top of `BareField`: **## What Field.h Adds Beyond BareField: | Feature | File Head...`**

file_006 - What role does HaloCells.h play in IPPL?

- Verdict: **``Incorrect``**
- Correct reference answer: Defines the halo-exchange machinery for fields: pack/unpack, boundary exchange, and periodic handling for ghost cells.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `2.98s`
- Missing/weak reference signals: `halo-exchange, machinery, fields, pack/unpack, boundary, exchange, periodic, handling`
- Claude Code answer excerpt: ``

file_007 - What is the purpose of FieldLayout.h?

- Verdict: **``Incorrect``**
- Correct reference answer: Defines how a global `NDIndex` domain is partitioned across MPI ranks, including local subdomains, neighbors, and communication ranges.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `205.78s`
- Missing/weak reference signals: `global, ndindex, domain, partitioned, across, mpi, ranks, local`
- Claude Code answer excerpt: ``

file_008 - What does ParticleBase.h define?

- Verdict: **``Correct``**
- Correct reference answer: Defines the generic particle container base class with positions, optional IDs, registered attributes, and a particle layout that controls migration.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.62`
- Latency: `41.43s`
- Matched reference signals: `particle, container, base, positions, ids, attributes, layout`
- Missing/weak reference signals: `generic, optional, registered, controls, migration`
- Claude Code answer excerpt: `on reading `/psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/src/Particle/ParticleBase.h`, here is what **``ParticleBase.h``** defines: **## Main Class Definition (lines 86-376) | What | Description |**

----- -----		
	`class ParticleBase< PLayout,	
IDProperties...>`	Template class that serves as t...	

file_009 - What is the purpose of ParticleSpatialLayout.h?

- Verdict: **``Correct``**
- Correct reference answer: Defines the spatial particle layout that decides ownership by region and migrates particles to the correct rank after motion.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`

- Latency: `17.50s`
- Matched reference signals: `spatial, particle, layout, region, particles, after`
- Missing/weak reference signals: `decides, ownership, migrates, correct, rank, motion`
- Claude Code answer excerpt: `on reading `data/raw/ippl/src/Particle/ParticleSpatialLayout.h`, here is the purpose of `ParticleSpatialLayout.h`: `## Purpose Summary` `ParticleSpatialLayout.h` declares a template class that implements `spatial decomposition-based particle distribution across MPI processors`. This layout strategy: 1. `Mai...`

file_010 - What does ParticleAttrib.h implement?

- Verdict: `Correct`
- Correct reference answer: Implements the `ParticleAttrib` storage object: a Kokkos-view-backed particle attribute with `resize`, `serialization`, and `mirror/access` helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.55`
- Reference identifier/location hits: `1/1`
- Latency: `24.44s`
- Matched reference signals: `implements, particleattrib, storage, object, particle, attribute, ParticleAttrib`
- Missing/weak reference signals: `kokkos-view-backed, resize, serialization, mirror/access, helpers`
- Claude Code answer excerpt: `on the files at `/psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/src/Particle/`, `ParticleAttrib.h` implements a templated class for all particle attribute classes in the IPPL library. The header provides: `## What ParticleAttrib.h Implements`: 1. `Primary Templated Class`: A template class `Par...`

file_011 - What is UniformCartesian.h for?

- Verdict: `Correct`
- Correct reference answer: Defines the uniform Cartesian mesh class with `spacing`, `origin`, `cell volume`, `total mesh volume`, and `vertex-position` helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.75`
- Latency: `21.31s`
- Matched reference signals: `uniform, cartesian, mesh, spacing, origin, cell, volume`
- Missing/weak reference signals: `total, vertex-position, helpers`
- Claude Code answer excerpt: `on my exploration of the repository, `UniformCartesian.h` defines a template class `ippl::UniformCartesian<T, Dim>` that represents `uniform-spacing Cartesian meshes`. `## Purpose and Key Features`: 1. `Inherits from`: [`Mesh<T, Dim>`](data/raw/ippl/src/Meshes/Mesh.h) – it's the specialized mesh base class for...

file_012 - What does OrthogonalRecursiveBisection.h do?

- Verdict: `Incorrect`
- Correct reference answer: Declares the orthogonal recursive bisection domain-decomposition/load-balancing class and its `cut/median/repartition` helpers.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `30.89s`
- Missing/weak reference signals: `orthogonal, recursive, bisection, domain-decomposition/load-balancing, cut/median/repartition, helpers`
- Claude Code answer excerpt: ``

file_013 - What is the purpose of CIC.h in the Interpolation subsystem?

- Verdict: `Correct`
- Correct reference answer: Declares the first-order cloud-in-cell interpolation helpers for `scatter/gather` between particles and grid fields.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.56`
- Latency: `23.24s`
- Matched reference signals: `first-order, cloud-in-cell, interpolation, particles, grid`
- Missing/weak reference signals: `helpers, scatter/gather, between, fields`
- Claude Code answer excerpt: `Purpose of `CIC.h` (Cloud-In-Cell Interpolation) `Summary`: The file `/psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/src/Interpolation/CIC.hpp:3` implements `first-order Cloud-in-Cell (CIC)` grid interpolation used in the IPPL simulation framework. CIC is a common technique for scattering particl...

Definition Location

loc_001 - Where is FFT initialization implemented in IPPL?

- Verdict: ****Partial****
- Correct reference answer: FFT initialization lives in ``src/FFT/FFT.hpp``, especially ``FFTBase`` construction/setup and the transform-specific constructors that build the `heFFTe` plans.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.17``
- Reference identifier/location hits: ``1/3``
- Latency: ``79.72s``
- Matched reference signals: ``fft, heffte, FFT.hpp``
- Missing/weak reference signals: ``initialization, lives, src/fft/fft.hpp, especially, fftbase, construction/setup, transform-specific, constructors``
- Claude Code answer excerpt: ``at your FFT.hpp file and the codebase, this uses heFFTe (https://github.com/ECP-copa/heFFTe) as an external backend library for high-performance distributed FFT operations on Kokkos views. The IPPL FFT class acts as a thin interface between heffte's lower-level routines and IPPL's field types with ghost layers....``

loc_002 - Where does IPPL set up the MPI environment?

- Verdict: ****Incorrect****
- Correct reference answer: IPPL sets up MPI in ``src/Ippl.cpp::initialize()`` by constructing ``mpi::Environment`` and ``mpi::Communicator``; the low-level ``MPI_Init`` check/call is in ``src/Communicate/Environment.cpp``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/9``
- Latency: ``12.25s``
- Missing/weak reference signals: ``sets, mpi, src/ippl.cpp::initialize, constructing, mpi::environment, mpi::communicator, low-level, mpi_init``
- Claude Code answer excerpt: ```

loc_003 - Where is Kokkos initialized and finalized inside IPPL?

- Verdict: ****Correct****
- Correct reference answer: Kokkos is initialized and finalized directly in ``src/Ippl.cpp`` inside ``ippl::initialize()`` and ``ippl::finalize()``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.38``
- Reference identifier/location hits: ``6/8``
- Latency: ``25.01s``
- Matched reference signals: ``kokkos, ippl::initialize, ippl::finalize, Ippl.cpp, finalize(), ippl::finalize(), src/Ippl.cpp``
- Missing/weak reference signals: ``initialized, finalized, directly, src/ippl.cpp, inside, initialize(), ippl::initialize()``
- Claude Code answer excerpt: ``~/data/raw/ippl/src/Ippl.cpp: | Function | Location of Kokkos call | Line |
|-----|-----|-----| | ippl::initialize(int& argc, char* argv[], MPI_Comm comm)` |
Kokkos::initialize(argc, argv) after argument parsing and before exception handling cleanup | 99 | |
ippl::finalize()` | **Kokk...`

loc_004 - Where is the orthogonal recursive bisection load balancer implemented?

- Verdict: ****Partial****
- Correct reference answer: The ORB load balancer is implemented in ``src/Decomposition/OrthogonalRecursiveBisection.hpp`` with declarations in the matching header.
- Explanation: The answer is related, but it misses the concrete implementation location or symbol emphasized by the Codex reference.
- Reference-keyword coverage: ``0.43``
- Reference identifier/location hits: ``0/2``
- Latency: ``2.45s``
- Matched reference signals: ``orb, load, balancer``
- Missing/weak reference signals: ``src/decomposition/orthogonalrecursivebisection.hpp, declarations, matching, header, OrthogonalRecursiveBisection.hpp, src/Decomposition/OrthogonalRecursiveBisection.hpp``
- Claude Code answer excerpt: ``ll search through the repository codebase to find where the orthogonal recursive bisection (ORB) load balancer is implemented. Let me start by exploring the relevant parts of the IPPL source tree in `data/raw/ippl`.``

loc_005 - Where are boundary conditions applied to a Field?

- Verdict: ****Incorrect****
- Correct reference answer: Field BCs are attached in ``Field::setFieldBC()`` and actually applied through ``BConds::apply()`` and the concrete face classes in ``BcTypes.hpp``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related

implementation details.

- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/7`
- Latency: `3.41s`
- Missing/weak reference signals: `field, bcs, attached, field::setfieldbc, actually, applied, through, bconds::apply`
- Claude Code answer excerpt: ``

loc_006 - Where is the CIC (cloud-in-cell) scatter implemented?

- Verdict: ****Correct****
- Correct reference answer: CIC scatter is implemented in `src/Interpolation/CIC.hpp`, primarily `scatterToPoint()` and `scatterToField()`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.67`
- Reference identifier/location hits: `2/4`
- Latency: `19.94s`
- Matched reference signals: `cic, scatter, scatterToPoint, scatterToField, CIC.hpp, src/Interpolation/CIC.hpp`
- Missing/weak reference signals: `src/interpolation/cic.hpp, primarily, scatterToField(), scatterToPoint()`
- Claude Code answer excerpt: ****CIC (cloud-in-cell)**** interpolation is implemented in: - ****`data/raw/ippl/src/Interpolation/CIC.hpp`**** – Contains the full template implementations of `scatterToPoint`, `scatterToField`, and related gather functions for first-order cloud-in-cell grid interpolation. - ****`data/raw/ippl/src/Interpolation/CIC.h`**** – H...

loc_007 - Where is the CIC gather implemented?

- Verdict: ****Correct****
- Correct reference answer: CIC gather is implemented in `src/Interpolation/CIC.hpp`, primarily `gatherFromPoint()` and `gatherFromField()`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.67`
- Reference identifier/location hits: `2/4`
- Latency: `26.45s`
- Matched reference signals: `cic, gather, gatherFromPoint, gatherFromField, CIC.hpp, src/Interpolation/CIC.hpp`
- Missing/weak reference signals: `src/interpolation/cic.hpp, primarily, gatherFromField(), gatherFromPoint()`
- Claude Code answer excerpt: `on my exploration of the IPPL repository codebase: **\*\*The CIC gather is implemented in:\*\*** `data/raw/ippl/src/Interpolation/CIC.hpp` and declared in `data/raw/ippl/src/Interpolation/CIC.h` Specifically, it consists of two template functions implementing Cloud-in-Cell (CIC) interpolation gathering: 1. **\*\*`gatherFromPoin...`**

loc_008 - Where is halo exchange for BareField implemented?

- Verdict: ****Incorrect****
- Correct reference answer: Halo exchange for `BareField` is surfaced in `src/Field/BareField.hpp` and actually performed in `src/Field/HaloCells.hpp`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/5`
- Latency: `111.74s`
- Missing/weak reference signals: `halo, exchange, barefield, surfaced, src/field/barefield.hpp, actually, performed, src/field/halocells.hpp`
- Claude Code answer excerpt: ``

loc_009 - Where are the FFT-based open-boundary Poisson solver routines defined?

- Verdict: ****Incorrect****
- Correct reference answer: The FFT open-boundary Poisson routines are defined in `src/PoissonSolvers/FFTOpenPoissonSolver.h` and `src/PoissonSolvers/FFTOpenPoissonSolver.hpp`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/4`
- Latency: `11.80s`
- Missing/weak reference signals: `fft, open-boundary, poisson, defined, src/poissonsolvers/fftopenpoissonsolver.h, src/poissonsolvers/fftopenpoissonsolver.hpp, FFTOpenPoissonSolver.h, FFTOpenPoissonSolver.hpp`
- Claude Code answer excerpt: ``

loc_010 - Where is the conjugate gradient solver algorithm implemented?

- Verdict: ****Partial****
- Correct reference answer: The CG/PCG algorithm implementation is in `src/LinearSolvers/PCG.h`; the Poisson wrapper that instantiates it is `src/PoissonSolvers/PoissonCG.h`.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.25`
- Reference identifier/location hits: `2/4`
- Latency: `19.79s`
- Matched reference signals: `algorithm, implementation, PCG.h, src/LinearSolvers/PCG.h`
- Missing/weak reference signals: `cg/pcg, src/linearsolvers/pcg.h, poisson, wrapper, instantiates, src/poissonsolvers/poissoncg.h, PoissonCG.h, src/PoissonSolvers/PoissonCG.h`
- Claude Code answer excerpt: ****conjugate gradient (CG) solver**** is implemented in: ```data/raw/ippl/src/LinearSolvers/PCG.h``` This header file declares and implements two templated classes: 1. ****ippl::CG<...>**** – The base conjugate gradient algorithm class, with operator implementations at lines 40-182 (for general field types) and lines 25...

loc_011 - Where is the FEM Poisson solver implemented?

- Verdict: ****Correct****
- Correct reference answer: The FEM Poisson solver is implemented in `src/PoissonSolvers/FEMPoissonSolver.h`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.75`
- Reference identifier/location hits: `2/2`
- Latency: `14.79s`
- Matched reference signals: `fem, poisson, solver, FEMPoissonSolver.h, src/PoissonSolvers/FEMPoissonSolver.h`
- Missing/weak reference signals: `src/poissonsolvers/fempoissonsolver.h`
- Claude Code answer excerpt: ****FEM (Finite Element Method) Poisson solver**** `ippl::FEMPoissonSolver`` is implemented at: ```data/raw/ippl/src/PoissonSolvers/FEMPoissonSolver.h``` This header file defines a templated class (`line 21-196`) that inherits from `Poisson<FieldLHS, FieldRHS>` and implements the finite element method solver using Conj...

loc_012 - Where is the FDTD Maxwell solver base class defined?

- Verdict: ****Incorrect****
- Correct reference answer: The FDTD Maxwell solver base class is defined in `src/MaxwellSolvers/FDTSolverBase.h`` with implementation in `FDTSolverBase.hpp``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/3`
- Latency: `4.75s`
- Missing/weak reference signals: `fdtd, maxwell, solver, base, defined, src/maxwellsolvers/fdtdsolverbase.h, implementation, fdtdsolverbase.hpp`
- Claude Code answer excerpt: ``

loc_013 - Where is the LagrangeSpace class defined?

- Verdict: ****Incorrect****
- Correct reference answer: The `LagrangeSpace`` class is defined in `src/FEM/LagrangeSpace.h`` with methods implemented in `src/FEM/LagrangeSpace.hpp``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/5`
- Latency: `26.27s`
- Missing/weak reference signals: `lagrangespace, defined, src/fem/lagrangespace.h, src/fem/lagrangespace.hpp, LagrangeSpace, LagrangeSpace.h, LagrangeSpace.hpp, src/FEM/LagrangeSpace.h`
- Claude Code answer excerpt: ```ll continue from where the token limit cut off. The last complete action was checking for existing memory files. Let me split this work into manageable pieces: I already know there are 48 questions being generated, and memories #3-6 were written covering authentication flow, model selection interface, parallel exec...``

loc_014 - Where is the particle update routine (that moves particles across ranks) implemented?

- Verdict: ****Incorrect****
- Correct reference answer: Particle migration between ranks is implemented in `ParticleSpatialLayout::update()`` in `src/Particle/ParticleSpatialLayout.hpp``.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `118.93s`
- Recorded error: `exit 1``
- Missing/weak reference signals: `particle, migration, between, ranks, particlespatiallayout::update,

src/particle/particlespatiallayout.hpp`

- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

loc_015 - Where are IPPL timers defined?

- Verdict: ****Incorrect****

- Correct reference answer: IPPL timers are defined by the singleton timing layer in `src/Utility/IpplTimings.h/.cpp`, not just the low-level `Timer` helper.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.00`

- Reference identifier/location hits: `0/3`

- Latency: `3.48s`

- Missing/weak reference signals: `timers, defined, singleton, timing, layer, src/utility/ippltimings.h/.cpp, not, just`

- Claude Code answer excerpt: ``

Class Responsibility

cls_001 - What is the main responsibility of the BareField class?

- Verdict: ****Correct****

- Correct reference answer: `BareField` is the core distributed field storage class: it owns the data view, domain metadata, layout link, and halo operations.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.23`

- Reference identifier/location hits: `1/1`

- Latency: `15.43s`

- Matched reference signals: `barefield, field, view, BareField`

- Missing/weak reference signals: `core, distributed, storage, owns, domain, metadata, layout, link`

- Claude Code answer excerpt: `on my exploration of the IPPL repository for `BareField`, I can provide you an authoritative answer: **\*\*The main responsibility of the `BareField` class is to represent a real field.\*\*** This is explicitly stated in both places within the header file (`data/raw/ippl/src/Field/BareField.h`): 1. ****Line 3 comment****: ``cp...`

cls_002 - What is the main responsibility of the Field class compared to BareField?

- Verdict: ****Incorrect****

- Correct reference answer: `Field` extends `BareField` with a mesh, boundary-condition container, and mesh-aware integral/average semantics.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.00`

- Reference identifier/location hits: `0/2`

- Latency: `28.70s`

- Missing/weak reference signals: `field, extends, barefield, mesh, boundary-condition, container, mesh-aware, integral/average`

- Claude Code answer excerpt: ``

cls_003 - What does the FieldLayout class do and what are its main template parameters?

- Verdict: ****Incorrect****

- Correct reference answer: `FieldLayout<Dim>` distributes a global `NDIndex<Dim>` over ranks and stores local domains, neighbors, and communication ranges; its only template parameter is `Dim`.

- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.

- Reference-keyword coverage: `0.00`

- Latency: `87.84s`

- Recorded error: `exit 1`

- Missing/weak reference signals: `fieldlayout, dim, distributes, global, ndindex, dim, ranks, stores`

- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

cls_004 - What is the purpose of the ParticleBase class and how is it meant to be used?

- Verdict: ****Incorrect****

- Correct reference answer: `ParticleBase` is the user-extensible base container for particles; derived classes add attributes and register them so layout-driven migration can move them.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.00`

- Reference identifier/location hits: `0/1`

- Latency: `29.18s`
- Missing/weak reference signals: `particlebase, user-extensible, base, container, particles, derived, classes, attributes`
- Claude Code answer excerpt: ``

cls_005 - What does the ParticleSpatialLayout class do?

- Verdict: ****Correct****
- Correct reference answer: `ParticleSpatialLayout` decides which spatial region/rank owns each particle and performs the migration/update workflow after particles move.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.23`
- Reference identifier/location hits: `1/1`
- Latency: `2.90s`
- Matched reference signals: `particlespatiallayout, spatial, particle, ParticleSpatialLayout`
- Missing/weak reference signals: `decides, region/rank, owns, each, performs, migration/update, workflow, after`
- Claude Code answer excerpt: `I'll search through the IPPL repository for information about the `ParticleSpatialLayout` class. Let me start by exploring the relevant files in the `data/raw/ippl` directory structure related to particle classes and their spatial layout responsibilities.`

cls_006 - What is the responsibility of UniformCartesian?

- Verdict: ****Incorrect****
- Correct reference answer: `UniformCartesian` represents a uniform Cartesian mesh and exposes spacing/origin/volume and vertex-position geometry queries.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `93.12s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `uniformcartesian, represents, uniform, cartesian, mesh, exposes, spacing/origin/volume, vertex-position`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

cls_007 - What does the Mesh base class define?

- Verdict: ****Incorrect****
- Correct reference answer: `Mesh` is the abstract base interface for meshes: origin, grid size, spacing, vertex positions, and volume queries.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `12.41s`
- Missing/weak reference signals: `mesh, abstract, base, interface, meshes, origin, grid, size`
- Claude Code answer excerpt: ``

cls_008 - What is the responsibility of the Communicator class in ippl::mpi?

- Verdict: ****Incorrect****
- Correct reference answer: `ippl::mpi::Communicator` wraps MPI point-to-point, collectives, tag handling, and managed communication buffers.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `2.62s`
- Missing/weak reference signals: `ippl::mpi::communicator, wraps, mpi, point-to-point, collectives, tag, handling, managed`
- Claude Code answer excerpt: `I'll explore the repository to understand what the `communicator` class does, though I note from CLAUDE.md that the raw IPPL tree is not in git – so any files found there are local or generated artifacts. Let me first check whether an IPPL source file exists under `data/raw/ippl`.`

cls_009 - What is the role of the Environment class in ippl::mpi?

- Verdict: ****Incorrect****
- Correct reference answer: `ippl::mpi::Environment` manages MPI environment lifetime checks/teardown and provides an abort hook.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`

- Reference identifier/location hits: `0/1`
- Latency: `26.22s`
- Missing/weak reference signals: `ippl::mpi::environment, manages, mpi, environment, lifetime, checks/teardown, abort, hook`
- Claude Code answer excerpt: ``

cls_010 - What does the Archive class do in the Communicate module?

- Verdict: ****Incorrect****
- Correct reference answer: `Archive` serializes and deserializes Kokkos views and vector-valued views into raw byte buffers for MPI transfer.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `6.03s`
- Missing/weak reference signals: `archive, serializes, deserializes, kokkos, views, vector-valued, raw, byte`
- Claude Code answer excerpt: ``

cls_011 - What is the role of the BConds class?

- Verdict: ****Correct****
- Correct reference answer: `BConds` is the container of boundary-condition faces and orchestrates neighbor discovery, BC application, and ghost-to-physical assignment.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.40`
- Reference identifier/location hits: `1/1`
- Latency: `25.68s`
- Matched reference signals: `bconds, container, faces, neighbor, BConds`
- Missing/weak reference signals: `boundary-condition, orchestrates, discovery, application, ghost-to-physical, assignment`
- Claude Code answer excerpt: `on my analysis of the codebase, here is what I found about the **BConds** class:
Role of BConds Class The `ippl::BConds` class serves as a **container for field boundary conditions** primary role and responsibilities are documented in its header files at
`[src/Field/BConds.h](data/raw/ippl/src/Field/BConds....``

cls_012 - What is the role of the BaseManager class?

- Verdict: ****Incorrect****
- Correct reference answer: `BaseManager` provides the simulation loop skeleton: `pre\_run`, `pre\_step`, `advance`, `post\_step`, and `run(nt)`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/6`
- Latency: `4.24s`
- Missing/weak reference signals: `basemanager, simulation, loop, skeleton, pre\_run, pre\_step, advance, post\_step`
- Claude Code answer excerpt: ``

cls_013 - What does the Index class represent and what operations does it support?

- Verdict: ****Correct****
- Correct reference answer: `Index` represents a regular strided integer range/slice and supports range-style arithmetic and comparison operations used in domain descriptions.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `1/1`
- Latency: `43.21s`
- Matched reference signals: `index, integer, supports, arithmetic, operations, Index`
- Missing/weak reference signals: `represents, regular, strided, range/slice, range-style, comparison, domain, descriptions`
- Claude Code answer excerpt: `Index class from the IPPL library defines a **slice representation** for arrays/vectors using first-element (`a`), last-indexed value (`b`) semantics `[[src](file:///psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/src/Index/Index.h)](Index.h)`, along with an integer `stride_s` (step) that can be positiv...`

Algorithm

alg_001 - How does the FFT-based open-boundary Poisson solver compute the electrostatic potential?

- Verdict: ****Incorrect****
- Correct reference answer: The FFT open-boundary solver embeds/extends the RHS onto larger domains, FFTs both the density and a precomputed Green's function, multiplies in Fourier space, inverse transforms, and optionally computes gradients/Hessians.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `8.21s`
- Missing/weak reference signals: `fft, open-boundary, solver, embeds/extends, rhs, onto, larger, domains`
- Claude Code answer excerpt: ``

alg_002 - How does the FFT-based periodic Poisson solver work in IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: The periodic FFT solver FFTs the charge density, divides each Fourier mode by $|k|^2$ (with the zero mode handled separately), then inverse transforms; gradients are computed by multiplying by $ik/|k|^2$ before the inverse FFT.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/2`
- Latency: `10.72s`
- Missing/weak reference signals: `periodic, fft, solver, ffts, charge, density, divides, each`
- Claude Code answer excerpt: ``

alg_003 - How does the truncated Green periodic Poisson solver differ from the standard periodic c

- Verdict: ****Incorrect****
- Correct reference answer: The truncated-Green periodic solver replaces the simple $1/|k|^2$ kernel with the FFT of a truncated/erf-smoothed Green's function $\text{forceConstant} * \text{erf}(\alpha r) / r$, then follows the same transform-multiply-inverse pattern.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `189.82s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `truncated-green, periodic, solver, replaces, simple, kernel, fft, truncated/erf-smoothed`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

alg_004 - How does IPPL's CG solver iterate in the PCG.h implementation?

- Verdict: ****Correct****
- Correct reference answer: In `PCG.h`, the CG iteration forms residual/direction fields, applies the operator, computes `alpha` and `beta` from inner products, updates the solution/residual/direction, and stops on tolerance or iteration count; the preconditioned variant inserts M^{-1} applications on the residual.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.35`
- Reference identifier/location hits: `3/4`
- Latency: `27.46s`
- Matched reference signals: `iteration, operator, alpha, beta, tolerance, preconditioned, residual, PCG.h`
- Missing/weak reference signals: `pcg.h, forms, residual/direction, fields, applies, computes, inner, products`
- Claude Code answer excerpt: `on `data/raw/ippl/src/LinearSolvers/PCG.h`, here's how IPPL's CG solver iterates in the **preconditioned conjugate gradient (PCG)** implementation: **## Core Iteration Loop (lines 471-495)** ```cpp while (this->iterations_m < maxIterations && this->residueNorm > tolerance) { q = this->op_m(d); // Apply matrix to precondition...`

alg_005 - How is cloud-in-cell interpolation performed in IPPL (both scatter and gather directions

- Verdict: ****Incorrect****
- Correct reference answer: CIC computes lower/upper weights per axis, scatters with atomic adds to the 2^{Dim} neighboring grid points, and gathers by summing the same weighted stencil back to particles.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.05`
- Reference identifier/location hits: `0/1`
- Latency: `55.17s`
- Matched reference signals: `dim`
- Missing/weak reference signals: `cic, computes, lower/upper, weights, per, axis, scatters, atomic`
- Claude Code answer excerpt: `add new attribute slots (like energy bins) in your Cppnetlib-based IPPL system: 1. **Locate** `ParticleAttrib<T>` and its instantiation within a struct like this: ```cpp typedef ippl::Vector<double, Dim> position_type; // Your particle attributes: R is positions, add others here (e.g., E0`

for energy bins) ParticleAt...`

alg_006 - How does the standard FDTD solver advance the electromagnetic field in time?

- Verdict: ****Incorrect****
- Correct reference answer: The standard FDTD solver advances the source-like field with a second-order finite-difference update using the previous and current time levels plus neighboring cells and source terms, then fills halos and reapplies BCs.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `44.56s`
- Missing/weak reference signals: `standard, ftdt, solver, advances, source-like, field, second-order, finite-difference`
- Claude Code answer excerpt: ``

alg_007 - How does the LagrangeSpace class evaluate degrees of freedom per element?

- Verdict: ****Incorrect****
- Correct reference answer: `LagrangeSpace` evaluates element DOFs by mapping local element indices to global DOF indices, then evaluates order-1 reference-element shape functions and gradients on those local DOFs.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.05`
- Reference identifier/location hits: `0/1`
- Latency: `44.47s`
- Matched reference signals: `those`
- Missing/weak reference signals: `lagrangespace, evaluates, element, dofs, mapping, local, indices, global`
- Claude Code answer excerpt: `I've finished analyzing modelfiles! Let me give you a summary of what I found: ## Key Findings from Modelfile Analysis The main issue is that `litellm_ollama_config.yaml` references models like "gemma4-272b" which don't actually exist on disk. The actual model files have names with "\_cc" suffix (e.g., `Modelfile.qwen...`

alg_008 - How is Gauss-Jacobi quadrature implemented and used in IPPL FEM?

- Verdict: ****Incorrect****
- Correct reference answer: Gauss-Jacobi quadrature computes nodes/weights by Newton iteration on Jacobi polynomial roots, starting from Chebyshev or LehrFEM-style guesses, and uses the resulting rule on reference elements in FEM.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.17`
- Latency: `3.99s`
- Matched reference signals: `gauss-jacobi, quadrature, fem`
- Missing/weak reference signals: `computes, nodes/weights, newton, iteration, jacobi, polynomial, roots, starting`
- Claude Code answer excerpt: `I'll explore the repository to understand how Gauss-Jacobi quadrature is implemented and used in IPPL's Finite Element Method (FEM) context.`

alg_009 - How does the orthogonal recursive bisection decide where to cut the domain?

- Verdict: ****Correct****
- Correct reference answer: ORB chooses a cut axis from the domain geometry and then finds a weight-balanced cut/median so recursive splits distribute load more evenly.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Latency: `27.83s`
- Matched reference signals: `orb, chooses, cut, axis, domain, recursive, splits, load`
- Missing/weak reference signals: `geometry, then, finds, weight-balanced, cut/median, distribute, more, evenly`
- Claude Code answer excerpt: `on the code I read from [OrthogonalRecursiveBisection.hpp](data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.hpp), here's how **orthogonal recursive bisection (ORB)** decides where to cut a domain: ## The Decision Process The algorithm follows these steps in `binaryRepartition()` ([line 14](file:///psi/...`

alg_010 - How does inverse transform sampling generate particles from a given distribution in IPPL

- Verdict: ****Incorrect****
- Correct reference answer: Inverse transform sampling maps uniform random values in CDF space to samples by estimating the inverse and refining it with Newton-Raphson, dimension by dimension.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`

- Latency: `14.70s`
- Missing/weak reference signals: `inverse, transform, sampling, maps, uniform, random, values, cdf`
- Claude Code answer excerpt: ``

alg_011 - How does the FFT class interface with heFFTe to perform complex-to-complex and real-to-complex transforms?

- Verdict: ****Incorrect****
- Correct reference answer: The FFT layer builds heFFTe boxes/plans in `setup()`, copies IPPL field data into the required views/subviews, calls heFFTe forward/backward transforms, and copies results back for CC/RC/DST/DCT variants.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `13.33s`
- Missing/weak reference signals: `fft, layer, builds, heffte, boxes/plans, setup, copies, field`
- Claude Code answer excerpt: ``

alg_012 - How does IPPL's preconditioned CG use the preconditioner defined in Preconditioner.h?

- Verdict: ****Incorrect****
- Correct reference answer: PCG applies a chosen preconditioner from `Preconditioner.h` to the residual, uses the preconditioned residual in its inner products and direction updates, and iterates until tolerance/max iterations.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `6.58s`
- Missing/weak reference signals: `pcg, applies, chosen, preconditioner, preconditioner.h, residual, preconditioned, inner`
- Claude Code answer excerpt: ``

Data Flow

flow_001 - How does charge density flow from particles to the grid in a typical PIC step in IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: In the PIC workflow, particle charges and positions are deposited onto the grid with CIC (`scatter`) or FEM assembly (`assemble\_rhs\_from\_particles`), accumulated into `rho`, and then normalized/background-shifted in the manager before solving.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/3`
- Latency: `30.45s`
- Missing/weak reference signals: `pic, workflow, particle, charges, positions, deposited, onto, grid`
- Claude Code answer excerpt: ``

flow_002 - How does the electric field flow from the grid back to particles after a Poisson solve?

- Verdict: ****Incorrect****
- Correct reference answer: After the field solve, the manager gathers the electric field back to particles with CIC `gather(...)` or FEM `interpolate\_grad\_to\_diracs(...)` , storing particle-local `E` values for the push step.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/2`
- Latency: `8.51s`
- Missing/weak reference signals: `after, field, solve, manager, gathers, electric, back, particles`
- Claude Code answer excerpt: ``

flow_003 - How are halo cells exchanged between neighboring MPI ranks for a BareField?

- Verdict: ****Incorrect****
- Correct reference answer: Halo exchange uses `FieldLayout` neighbor/range metadata to pack field subviews, send/receive them with MPI, unpack into ghost or physical regions, and handle serial-periodic dimensions locally.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`

- Reference identifier/location hits: `0/1`
- Latency: `51.35s`
- Missing/weak reference signals: `halo, exchange, fieldlayout, neighbor/range, metadata, pack, field, subviews`
- Claude Code answer excerpt: ``

flow_004 - How are particles migrated between MPI ranks by ParticleSpatialLayout?

- Verdict: ****Incorrect****
- Correct reference answer: `ParticleSpatialLayout::update()` applies particle BCs, locates destination ranks, advertises receive counts with RMA, sends all registered attributes, destroys invalid local particles, and receives the incoming ones.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/3`
- Latency: `16.41s`
- Missing/weak reference signals: `particlespatiallayout::update, applies, particle, bcs, locates, destination, ranks, advertises`
- Claude Code answer excerpt: ``

flow_005 - How does FieldLayout decide which index ranges live on which rank?

- Verdict: ****Incorrect****
- Correct reference answer: `FieldLayout` uses the partitioner and the requested parallel/serial decomposition flags to assign each rank a local `NDIndex`, plus neighbor and communication range tables.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `49.65s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `fieldlayout, partitioner, requested, parallel/serial, decomposition, flags, assign, each`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

flow_006 - How is a ParameterList used to configure the FFT and solver classes?

- Verdict: ****Partial****
- Correct reference answer: `ParameterList` is merged into FFT and solver objects, then consumed by `setup()/initialize()/solve()` for communication mode, pencil/reorder flags, transform direction, solver type, tolerances, preconditioner settings, and output type.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.30`
- Reference identifier/location hits: `1/5`
- Latency: `172.50s`
- Matched reference signals: `parameterlist, fft, solver, then, setup, solve, ParameterList`
- Missing/weak reference signals: `merged, objects, consumed, initialize, communication, mode, pencil/reorder, flags`
- Claude Code answer excerpt: `me continue explaining the `initSolver()` pattern findings for your "any problem needing field solve setup" category: ## FieldSolver Pattern in data/raw/ippl/alpine/FieldSolver.hpp The existing code matches **exactly** what you're looking for (170-325): ``cpp void initFFTSolver() { // line 170 - FFT initialization...`

flow_007 - How does data flow through the BaseManager's pre_step / advance / post_step hooks during simulation?

- Verdict: ****Incorrect****
- Correct reference answer: `BaseManager::run()` executes `pre\_step()`, `advance()`, and `post\_step()` for each timestep; derived managers use those hooks for deposit/solve/gather, time updates, and output.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/6`
- Latency: `18.71s`
- Missing/weak reference signals: `basemanager::run, executes, pre\_step, advance, post\_step, each, timestep, derived`
- Claude Code answer excerpt: ``

flow_008 - How does the FEM solver move data between fields and FEMVector representations?

- Verdict: ****Incorrect****
- Correct reference answer: FEM solvers move data by assembling field data into FEM load/stiffness representations (`evaluateLoadVector`, `evaluateAx`, `FEMVector`) and then interpolating the solved field

quantities back where needed.

- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `56.14s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `fem, solvers, move, assembling, field, fem, load/stiffness, representations`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

flow_009 - How does OrthogonalRecursiveBisection redistribute particles after rebalancing?

- Verdict: ****Correct****
- Correct reference answer: After ORB recomputes the decomposition, particle ownership changes are realized by updating the layout/regions and then migrating particles with the usual particle update path.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.41`
- Latency: `31.44s`
- Matched reference signals: `after, orb, particle, changes, then, particles`
- Missing/weak reference signals: `recomputes, decomposition, ownership, realized, updating, layout/regions, migrating, usual`
- Claude Code answer excerpt: `Summary of how `OrthogonalRecursiveBisection` redistributes particles after rebalancing:\*\* The redistribution occurs during each loop iteration in [`binaryRepartition`])(data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.hpp#lines 53-102), which follows the ORB algorithm described by Demmel et al.: **### Redi...**`

Api Usage

api_001 - How do I create a 3D Field of doubles in IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: A typical 3D field is `Field<double, 3, UniformCartesian<double, 3>, Cell>` constructed from a `UniformCartesian` mesh and a `FieldLayout<3>`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/3`
- Latency: `11.08s`
- Missing/weak reference signals: `typical, field, double, uniformcartesian, cell, constructed, mesh, fieldlayout`
- Claude Code answer excerpt: ``

api_002 - How do I construct a UniformCartesian mesh with custom spacing and origin?

- Verdict: ****Incorrect****
- Correct reference answer: Construct the mesh as `UniformCartesian<double, Dim> mesh(domain, hx, origin);` where `hx` is the spacing vector and `origin` is the mesh origin.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/4`
- Latency: `16.32s`
- Missing/weak reference signals: `construct, mesh, uniformcartesian, double, dim, domain, origin, spacing`
- Claude Code answer excerpt: ``

api_003 - How do I attach a boundary condition (e.g. periodic) to a Field?

- Verdict: ****Incorrect****
- Correct reference answer: Create a `BConds<FieldT, Dim>`, assign per-face BC objects such as `PeriodicFace<FieldT>`, then call `field.setFieldBC(bc)`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.15`
- Reference identifier/location hits: `0/4`
- Latency: `134.14s`
- Matched reference signals: `create, call`
- Missing/weak reference signals: `bconds, fieldt, dim, assign, per-face, objects, such, periodicface`
- Claude Code answer excerpt: `Now create tool call documentation to explain the architecture and tie everything together`

api_004 - How do I define a custom particle class that adds mass and velocity attributes on top of ParticleBase?

- Verdict: ****Incorrect****
- Correct reference answer: Define a class derived from ``ParticleBase<ParticleSpatialLayout<...>>``, add ``mass`` and ``velocity`` attributes as members, and register them with ``addAttribute()`` in the constructor.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/4``
- Latency: ``4.69s``
- Missing/weak reference signals: ``define, derived, particlebase, particlespatiallayout, mass, velocity, attributes, members``
- Claude Code answer excerpt: ``

api_005 - How do I call `ippl::initialize` and `ippl::finalize` in a main program?

- Verdict: ****Incorrect****
- Correct reference answer: Call the free namespace functions ``ippl::initialize(argc, argv);`` and ``ippl::finalize();`` from ``main``, as shown in the docs/examples.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/7``
- Latency: ``26.16s``
- Missing/weak reference signals: ``call, free, namespace, ippl::initialize, argc, argv, ippl::finalize, main``
- Claude Code answer excerpt: ``

api_006 - How do I set up and run a forward FFT on an IPPL Field?

- Verdict: ****Correct****
- Correct reference answer: Set up the relevant layouts/fields/parameters, build an ``FFT<CCTransform, ...>`` or ``FFT<RCTransform, ...>`` object, and call ``transform(FORWARD, ...)`` on the field(s).
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.62``
- Reference identifier/location hits: ``0/4``
- Latency: ``17.29s``
- Matched reference signals: ``set, fft, cctransform, object, transform, forward, field``
- Missing/weak reference signals: ``relevant, layouts/fields/parameters, build, rctransform, call, FFT<CCTransform, ...>, FFT<RCTransform, ...>, field(s)``
- Claude Code answer excerpt: ``on my exploration of the repository, I found clear examples in **[doc/examples/BasicsFFT.hpp](file:///psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl/doc/examples/BasicsFFT.hpp)**. Here's how to set up and run a forward FFT: ## Setting Up an IPPL Field for Forward FFT ### 1. Define the Input Real Fie...``

api_007 - How do I construct an `FFTPeriodicPoissonSolver` and solve for the potential?

- Verdict: ****Incorrect****
- Correct reference answer: Construct ``FFTPeriodicPoissonSolver(lhs, rhs)`` (or default construct then set fields/params), ensure the RHS is attached, and call ``solve()`` to write the potential/gradient outputs.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/2``
- Latency: ``70.88s``
- Missing/weak reference signals: ``construct, fftperiodicpoissonsolver, lhs, rhs, default, then, set, fields/params``
- Claude Code answer excerpt: ``

api_008 - How do I invoke the conjugate gradient Poisson solver on a Field?

- Verdict: ****Incorrect****
- Correct reference answer: Use ``PoissonCG<FieldT>`` (or the Alpine ``FieldSolver`` wrapper around it), set its fields/parameters, and call ``solve()`` so the internal CG/PCG object iterates on the field.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/3``
- Latency: ``64.49s``
- Missing/weak reference signals: ``use, poissoncg, fieldt, alpine, fieldsolver, wrapper, around, set``
- Claude Code answer excerpt: ``'m asking permission: should I create answer files for the FieldContainer question (and similarly handle OpenPMD purposes) in their respective output directories?``

api_009 - How do I sample particles from a normal distribution using IPPL Random utilities?

- Verdict: ****Incorrect****
- Correct reference answer: For Gaussian draws, IPPL exposes both ``randn`` convenience generation and the more general ``InverseTransformSampling`` + ``NormalDistribution`` path.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/3``
- Latency: ``10.20s``
- Missing/weak reference signals: ``gaussian, draws, exposes, both, randn, convenience, generation, more``
- Claude Code answer excerpt: ``

api_010 - How do I register particle attributes with ParticleBase so that they are communicated on migration?

- Verdict: ****Correct****
- Correct reference answer: Register each particle attribute with ``addAttribute(...)``; migration then serializes and transfers all registered attributes automatically.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.62``
- Reference identifier/location hits: ``0/1``
- Latency: ``30.40s``
- Matched reference signals: ``register, particle, attribute, addattribute, migration, all, registered, attributes``
- Missing/weak reference signals: ``each, then, serializes, transfers, automatically, addAttribute(...)``
- Claude Code answer excerpt: ``on the IPPL source code and documentation: ## Answer To register particle attributes with `ParticleBase` so they are communicated on MPI domain redistribution (migration), you use the **`addAttribute()`** method of `ParticleBase`. ### Required Usage Pattern ``cpp // 1. Declare your attribute as a member or local va...``

api_011 - How do I access the MPI rank and size through the IPPL Communicator?

- Verdict: ****Incorrect****
- Correct reference answer: Access rank and size with ``ippl::Comm->rank()`` and ``ippl::Comm->size()`` (or the same methods on a ``Communicator`` object).
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/6``
- Latency: ``3.45s``
- Missing/weak reference signals: ``access, rank, size, ippl::comm-, same, communicator, object, Communicator``
- Claude Code answer excerpt: ``

api_012 - How do I use IpplTimings to measure a region of code?

- Verdict: ****Incorrect****
- Correct reference answer: Get a timer handle with ``IpplTimings::getTimer("name")``, surround the code with ``startTimer()`` / ``stopTimer()``, and print with ``IpplTimings::print()``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/8``
- Latency: ``5.65s``
- Missing/weak reference signals: ``get, timer, handle, ippltimings::gettimer, name, surround, starttimer, stoptimer``
- Claude Code answer excerpt: ``

Parallelism And Kokkos

par_001 - How does IPPL achieve performance portability across CPUs and GPUs?

- Verdict: ****Incorrect****
- Correct reference answer: IPPL gets performance portability by combining Kokkos execution/memory abstractions with MPI and heFFTe, so the same code paths can target CPUs and GPUs.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: ``0.00``
- Latency: ``745.35s``
- Recorded error: ``exit 1``
- Missing/weak reference signals: ``gets, performance, portability, combining, kokkos, execution/memory, abstractions, mpi``
- Claude Code answer excerpt: ``API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE_CODE_MAX_OUTPUT_TOKENS environment variable.``

par_002 - Why does IPPL use Kokkos::View for field storage?

- Verdict: ****Incorrect****
- Correct reference answer: IPPL uses `Kokkos::View` because fields need execution-space-aware storage, subviews, mirrors, and portable kernels across host/device memory spaces.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `204.70s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `kokkos::view, because, fields, need, execution-space-aware, storage, subviews, mirrors`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

par_003 - How does IPPL parallelize loops over field indices using Kokkos RangePolicy?

- Verdict: ****Incorrect****
- Correct reference answer: Loops are parallelized through `ippl::getRangePolicy` / `createRangePolicy` and the `ippl::parallel\_for` wrapper over Kokkos `RangePolicy`/`MDRangePolicy`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/5`
- Latency: `18.94s`
- Missing/weak reference signals: `loops, parallelized, through, ippl::getrangepolicy, createrangepolicy, ippl::parallel\_for, wrapper, kokkos`
- Claude Code answer excerpt: ``

par_004 - How does IPPL support mixed precision (float vs double) for fields?

- Verdict: ****Incorrect****
- Correct reference answer: Precision is chosen by the field value type (`Field<float,...>` vs `Field<double,...>`), and the codebase also contains mixed-precision examples where fields/particles deliberately use different precisions.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.06`
- Reference identifier/location hits: `0/2`
- Latency: `147.51s`
- Matched reference signals: `type`
- Missing/weak reference signals: `precision, chosen, field, value, float, double, codebase, contains`
- Claude Code answer excerpt: `on the repository analysis, here's the current state: ## Recent Changes Summary (last 3 commits) The last major change (`21590ce`) was work to fix issues with Claude Code evaluation pipeline. Key changes include: #### Modified Files & Impact: | File | Change Type | Purpose |
|-----|-----|-----| | `litell...`

par_005 - How does IPPL handle multiple execution spaces like OpenMP and CUDA in the same build?

- Verdict: ****Incorrect****
- Correct reference answer: Multiple execution spaces are enabled at build time through `IPPL\_PLATFORMS` and Kokkos/HeFFTe backend flags; IPPL also has container helpers for multi-space data.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `6.41s`
- Missing/weak reference signals: `multiple, execution, spaces, enabled, build, time, through, ippl\_platforms`
- Claude Code answer excerpt: ``

par_006 - What is the purpose of ParallelDispatch.h and what helpers does it expose?

- Verdict: ****Incorrect****
- Correct reference answer: `ParallelDispatch.h` wraps Kokkos policies and functors, exposing `RangePolicy`, `getRangePolicy`, `createRangePolicy`, `parallel\_for`, and `parallel\_reduce`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/6`
- Latency: `21.24s`
- Missing/weak reference signals: `paralleldispatch.h, wraps, kokkos, policies, functors, exposing, rangepolicy, getrangepolicy`
- Claude Code answer excerpt: ``

par_007 - How does the Archive class serialize/deserialize Kokkos views for MPI transfer?

- Verdict: ****Incorrect****
- Correct reference answer: `Archive` serializes Kokkos views into one-dimensional byte buffers with Kokkos kernels and `memcpy`, then deserializes them back into views on receive.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/2`
- Latency: `113.78s`
- Missing/weak reference signals: `archive, serializes, kokkos, views, one-dimensional, byte, buffers, kernels`
- Claude Code answer excerpt: `ll save a local evals summary first so I can continue with the rest of my plan:`

par_008 - How are MPI tags managed in IPPL to avoid collisions across subsystems?

- Verdict: ****Incorrect****
- Correct reference answer: MPI tags are managed with fixed base tags in `Tags.h` and the cyclic tag allocator in `TagMaker`, which `Communicator` inherits.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/3`
- Latency: `26.12s`
- Missing/weak reference signals: `mpi, tags, managed, fixed, base, tags.h, cyclic, tag`
- Claude Code answer excerpt: ``

par_009 - What does Kokkos::fence do, and when does IPPL call the wrapper ippl::fence?

- Verdict: ****Incorrect****
- Correct reference answer: `Kokkos::fence()` waits for outstanding kernels; IPPL exposes `ippl::fence()` as a runtime wrapper in `Ippl.cpp` and also fences internally after many kernel/data-movement steps.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/6`
- Latency: `27.01s`
- Missing/weak reference signals: `kokkos::fence, waits, outstanding, kernels, exposes, ippl::fence, runtime, wrapper`
- Claude Code answer excerpt: ``

Boundary And Halo

bc_001 - What boundary condition types are provided in BcTypes.h?

- Verdict: ****Correct****
- Correct reference answer: `BcTypes.h` provides periodic, constant, zero, extrapolating, and no-BC face classes.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `1/1`
- Latency: `27.13s`
- Matched reference signals: `periodic, constant, zero, BcTypes.h`
- Missing/weak reference signals: `bctypes.h, extrapolating, no-bc, face, classes`
- Claude Code answer excerpt: `on `data/raw/ippl/src/Field/BcTypes.h`, the following **field BC (boundary condition) types** are defined via the `FieldBC` enumeration: | Enum Value | Symbolic Constant | Bit Pattern | Description | |-----|-----|-----|-----| | `0b0000` | `PERIODI...`

bc_002 - How does a periodic boundary condition differ from a Neumann boundary condition in IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: Periodic BCs wrap values from the opposite side of the domain; there is no explicit `NeumannFace` class in this codebase, so the closest non-periodic behavior is constant/extrapolating face treatment.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.06`
- Reference identifier/location hits: `0/1`
- Latency: `49.47s`
- Matched reference signals: `there`
- Missing/weak reference signals: `periodic, bcs, wrap, values, opposite, side, domain, explicit`
- Claude Code answer excerpt: `need your authorization to proceed with writing memory files – would you like me`

to continue updating all Qwen 3x7B configs in their respective memories, or is there something else blocking the file write permission?`

bc_003 - How are ghost cells populated during a halo exchange?

- Verdict: ****Incorrect****
- Correct reference answer: Ghost cells are populated mainly by `fillHalo()` / `PeriodicFace::apply()`: source slabs are packed, exchanged, and unpacked into ghost regions; `accumulateHalo\_noghost()` is a different operation.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.19`
- Reference identifier/location hits: `0/5`
- Latency: `5.02s`
- Matched reference signals: `ghost, cells`
- Missing/weak reference signals: `populated, mainly, fillhalo, periodicface::apply, source, slabs, packed, exchanged`
- Claude Code answer excerpt: `I'll help you answer this benchmark question about ghost cells and halo exchanges in IPPL (Intel Performance Primitives Library). Let me first explore the repository structure and then examine the specific code that handles halo/ghost cells.`

bc_004 - How are physical-cell values copied into ghost cells for periodic boundary conditions?

- Verdict: ****Incorrect****
- Correct reference answer: Periodic ghost cells are filled from opposite-side physical cells in `applyPeriodicSerialDim()` and the multi-rank `PeriodicFace::apply()` path.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/4`
- Latency: `26.79s`
- Missing/weak reference signals: `periodic, ghost, cells, filled, opposite-side, physical, applyperiodicserialdim, multi-rank`
- Claude Code answer excerpt: ``

bc_005 - How does the absorbing boundary condition in the MaxwellSolvers module damp outgoing waves?

- Verdict: ****Incorrect****
- Correct reference answer: The absorbing Maxwell BC uses second-order Mur-style formulas with precomputed weights and previous/current/next boundary values plus tangential neighbors to damp outgoing waves.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `349.78s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `absorbing, maxwell, second-order, mur-style, formulas, precomputed, weights, previous/current/next`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

Numerical Meaning

num_001 - What does the FFT-based Poisson solver physically compute in terms of Coulomb's law?

- Verdict: ****Incorrect****
- Correct reference answer: The FFT Poisson solvers compute electrostatic potential (and optionally field/gradient) from charge density, i.e. the discretized Coulomb potential corresponding to $-\Delta\phi = \rho$.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `236.87s`
- Missing/weak reference signals: `fft, poisson, solvers, compute, electrostatic, potential, optionally, field/gradient`
- Claude Code answer excerpt: `permission blocks on file writes (intentional). Let me summarize and present you with my findings: --- ## **Qwen Model Testing Report** - Session Complete [] ### Executive Summary Evaluated qwen models through gwendolen_cc_eval pipeline across multiple quantization variants (*q4km, *q8, kvant0 etc.). **Critical findi...`

num_002 - What is the physical meaning of the fields advanced by the FDTD solver?

- Verdict: ****Incorrect****
- Correct reference answer: The Maxwell solvers evolve electromagnetic field data: electric and magnetic fields (E , B) driven by a source/four-current field.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `14.00s`
- Missing/weak reference signals: `maxwell, solvers, evolve, electromagnetic, field, electric, magnetic, fields`
- Claude Code answer excerpt: ``

num_007 - What are the numerical tradeoffs between the FFT and CG Poisson solvers in IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: FFT Poisson is fast and direct on structured uniform grids with compatible BCs but less flexible; CG/PCG is iterative and may be slower/convergence-limited, but it is more flexible for alternative discretizations, BCs, and preconditioning.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `198.67s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `fft, poisson, fast, direct, structured, uniform, grids, compatible`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

Build And Install

build_001 - Which third-party libraries does IPPL depend on, and what are they used for?

- Verdict: ****Incorrect****
- Correct reference answer: Core third-party dependencies are MPI for distributed communication, Kokkos for performance portability, heFFTe for FFT backends/plans, FFTW/CuFFT as optional FFT backends, and GoogleTest for tests.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `133.01s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `core, third-party, dependencies, mpi, distributed, communication, kokkos, performance`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

build_002 - What C++ standard does IPPL require, and why?

- Verdict: ****Incorrect****
- Correct reference answer: IPPL requires C++20, as stated in the README badge and the installation examples/configuration flags.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `18.55s`
- Missing/weak reference signals: `requires, c++20, stated, readme, badge, installation, examples/configuration, flags`
- Claude Code answer excerpt: ``

build_003 - How is IPPL configured with CMake to enable a GPU build?

- Verdict: ****Incorrect****
- Correct reference answer: GPU builds are enabled through CMake options like `IPPL\_PLATFORMS=CUDA` or `IPPL\_PLATFORMS="HIP;OPENMP"` plus matching Kokkos/architecture/backend flags (`Kokkos\_ARCH\_\*`, `CMAKE\_CUDA\_ARCHITECTURES`, `Kokkos\_ENABLE\_HIP`, `Heffte\_ENABLE\_ROCM`, etc.).
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `66.75s`
- Recorded error: `exit 1`
- Missing/weak reference signals: `gpu, builds, enabled, through, cmake, options, like, ippl\_platforms`
- Claude Code answer excerpt: `API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE\_CODE\_MAX\_OUTPUT\_TOKENS environment variable.`

build_004 - What options does INSTALLATION.md describe for configuring Kokkos for IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: `INSTALLATION.md` describes Kokkos version selection plus backend/architecture options used with IPPL builds, including CUDA and HIP/OpenMP examples and `IPPL\_PLATFORMS` combinations.
- Explanation: The Claude Code run failed for this question, so no usable benchmark answer was produced.
- Reference-keyword coverage: `0.00`
- Latency: `414.64s`

- Recorded error: ``exit 1``
- Missing/weak reference signals: ``installation.md, describes, kokkos, version, selection, backend/architecture, options, builds``
- Claude Code answer excerpt: ``API Error: Claude's response exceeded the 32000 output token maximum. To configure this behavior, set the CLAUDE_CODE_MAX_OUTPUT_TOKENS environment variable.``

build_005 - How do I install IPPL on the EULER cluster according to the documentation?

- Verdict: ****Incorrect****
- Correct reference answer: The EULER install guide says to connect with VPN/SSH, switch to the new software stack, purge modules, load GCC/CMake/CUDA/OpenMPI, clone the repo, configure a build, compile, and test on an interactive node.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.05``
- Latency: ``7.17s``
- Matched reference signals: ``euler``
- Missing/weak reference signals: ``install, guide, says, connect, vpn/ssh, switch, new, software``
- Claude Code answer excerpt: ``ll explore the repository to find information about installing IPPL on the EULER cluster.``

Testing And Workflow

test_002 - What naming conventions does WORKFLOW.md require for variables, constants, and member v

- Verdict: ****Correct****
- Correct reference answer: ``WORKFLOW.md`` requires camelCase variables, CAPITAL_CASE compile-time constants, and member names suffixed with ``_m``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.70``
- Reference identifier/location hits: ``2/2``
- Latency: ``54.57s``
- Matched reference signals: ``workflow.md, requires, variables, compile-time, constants, member, suffixed, WORKFLOW.md``
- Missing/weak reference signals: ``camelcase, capital_case, names``
- Claude Code answer excerpt: ``on my search of the `data/raw/ippl/WORKFLOW.md` file, here are the naming conventions it requires: **Answer for test_002 (test_naming_conventions):** The WORKFLOW.md file specifies the following naming conventions at lines 1-5 (`data/raw/ippl/WORKFLOW.md:# Naming Conventions`): | Element Type | Convention | |-----|``

test_003 - When should code use Kokkos::sqrt versus std::sqrt according to WORKFLOW.md?

- Verdict: ****Correct****
- Correct reference answer: Use ``std::sqrt`` in host-only code, and ``Kokkos::sqrt`` in device-capable code such as ``KOKKOS_INLINE_FUNCTION`` kernels.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.75``
- Reference identifier/location hits: ``3/3``
- Latency: ``25.76s``
- Matched reference signals: ``use, std::sqrt, host-only, kokkos::sqrt, kokkos_inline_function, kernels, KOKKOS_INLINE_FUNCTION, Kokkos::sqrt``
- Missing/weak reference signals: ``device-capable, such``
- Claude Code answer excerpt: ``on **WORKFLOW.md** in the IPPL source tree (`data/raw/ippl/WORKFLOW.md:9-10`), here's when code should use `Kokkos::sqrt` versus `std::sqrt`: | Context | Use | Reason | |-----|-----|-----| **Host-only code** (CPU-execution on the host machine) | `std::sqrt` | Standard library is appropriate for CPU-bound,...``

test_004 - How does IPPL profile MPI calls using mpiP?

- Verdict: ****Correct****
- Correct reference answer: IPPL profiles MPI calls with mpiP by preloading ``libmpiP.so``, optionally setting ``MPIP`` flags, then parsing the produced ``_mpiP`` output with the bundled ``scripts/mpiP.py``.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.47``
- Reference identifier/location hits: ``5/5``
- Latency: ``41.73s``
- Matched reference signals: ``profiles, mpi, calls, mpip, libmpip.so, output, MPIP, libmpiP.so``
- Missing/weak reference signals: ``preloading, optionally, setting, flags, then, parsing, produced, bundled``
- Claude Code answer excerpt: ``on my exploration of the IPPL repository documentation, here's how IPPL profiles MPI calls using **mpiP**: **Overview IPPL uses the **[mpiP](https://github.com/LLNL/mpiP)** tool to collect``

statistics about MPI library calls in its applications. The relevant information is documented in [README.md](data/raw/ippl/d...`

test_005 - What does the SLURM job template for Gwendolen in the README look like?

- Verdict: ****Correct****
- Correct reference answer: The README does include a Gwendolen SLURM template showing the `gmerlin6` cluster, `gwendolen` partition, GPU count, and `srun ./<your\_executable> <args> --kokkos-map-device-id-by=mpi\_rank`.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.81`
- Reference identifier/location hits: `3/3`
- Latency: `49.36s`
- Matched reference signals: `gwendolen, slurm, template, gmerlin6, cluster, partition, gpu, srun`
- Missing/weak reference signals: `readme, showing, count`
- Claude Code answer excerpt: `SLURM job template for Gwendolen in `data/raw/ippl/README.md` (lines 111-128) looks like this: ``bash #!/bin/bash #SBATCH --time=00:05:00 # Define max time job will run (e.g. here 5 mins) #SBATCH --nodes=1 # No. of nodes (there is only 1 node on Gwendolen) #SBATCH --ntasks=4 # No. of tasks (max. 8) #SBATCH --cluste...`